Final Model Comparison Report

Summary: This report documents the 'research values' (best params, F1, ROC AUC, confusion matrix, classification report) for each algorithm run with Grid Search on the CKD dataset.

1) Summary table of key metrics

Algorithm	Best Parameters	F1 Score	ROC AUC
Logistic Regression	{}:".format(grid.best_params_),f1_macro) print("The confusion Matrix:\n",cm) from sklearn.metrics import roc_auc_score roc_auc_score(y_test,grid.predict_proba(X_test)[:,1]) table=pd.DataFrame.from_dict(re) # print classification report from sklearn.metrics import classification_report clf_report = classification_report(y_test, grid_predictions) print(clf_report) The f1_macro value for best parameter {'C': 10, 'penalty': 'l2', 'solver': 'lbfgs'}	0.9850	

SVM (Grid)	{}:".format(grid.best_params_),f1_macro)
	
	print("The confusion Matrix:\n",cm)
	
	from sklearn.metrics import roc_auc_score
	
	# ROC-AUC Score (for binary classification only)
	try:
	  roc = roc_auc_score(y_test, grid.predict_proba(X_test)[:, 1])
	  print("\nROC AUC Score: {:.4f}".format(roc))
	except Exception as e:
	  print("\nROC AUC could not be computed (likely non-binary classification).")
	
	table=pd.DataFrame.from_dict(re)
	
	# print classification report
	from sklearn.metrics import classification_report
	clf_report = classification_report(y_test, grid_predictions)
	
	print(clf_report)
	
	The f1_macro value for best parameter {'C': 10, 'degree': 2, 'gamma': 'scale', 'kernel': 'sigmoid
		0.9925	1.0000

Decision Tree (Grid)	{}:".format(grid.best_params_),f1_macro) print("The confusion Matrix:\n",cm) print("The report:\n",clf_report) The f1_macro value for best parameter {'criterion': 'entropy', 'max_features': 'sqrt', 'splitter': 'ra	0.9925	
Random Forest (Grid)	{}:".format(grid.best_params_),f1_macro) print("The confusion Matrix:\n",cm) print("The report:\n",clf_report) The f1_macro value for best parameter {'criterion': 'gini', 'max_features': 'log2', 'n_estimators':	0.9925	
SVM (Grid) (alt)			

2) Detailed results per algorithm

Logistic Regression

Best Parameters: {}:".format(grid.best_params_),f1_macro) print("The confusion Matrix:\n",cm) from sklearn.metrics import roc_auc_score roc_auc_score(y_test,grid.predict_proba(X_test)[:,1]) table=pd.DataFrame.from_dict(re) # print classification report from sklearn.metrics import classification_report clf_report = classification_report(y_test, grid_predictions) print(clf_report) The

f1_macro value for best parameter {'C': 10, 'penalty': 'l2', 'solver': 'lbfgs'}
F1 (reported): 0.9850141736106648
Confusion Matrix: [[51 0] [2 80]]
Classification Report:

```
precision    recall  f1-score   support

          no       0.96      1.00      0.98        51
         yes       1.00      0.98      0.99        82

    accuracy                           0.98       133
   macro avg       0.98      0.99      0.98       133
weighted avg
```

SVM (Grid)

Best Parameters: {}:''.format(grid.best_params_),f1_macro) print("The confusion Matrix:\n",cm) from sklearn.metrics import roc_auc_score # ROC-AUC Score (for binary classification only) try: roc = roc_auc_score(y_test, grid.predict_proba(X_test)[:, 1]) print("\nROC AUC Score: {:.4f}".format(roc)) except Exception as e: print("\nROC AUC could not be computed (likely non-binary classification).") table=pd.DataFrame.from_dict(re) # print classification report from sklearn.metrics import classification_report clf_report = classification_report(y_test, grid_predictions) print(clf_report) The f1_macro value for best parameter {'C': 10, 'degree': 2, 'gamma': 'scale', 'kernel': 'sigmoid'}
F1 (reported): 0.9924946382275899
ROC AUC: 1.0
Confusion Matrix: [[51 0] [1 81]]
Classification Report:

```
precision    recall  f1-score   support

          no       0.98      1.00      0.99        51
         yes       1.00      0.99      0.99        82

    accuracy                           0.99       133
   macro avg       0.99      0.99      0.99       133
weighted avg
```

Decision Tree (Grid)

Best Parameters: {}:''.format(grid.best_params_),f1_macro) print("The confusion Matrix:\n",cm) print("The report:\n",clf_report) The f1_macro value for best parameter {'criterion': 'entropy', 'max_features': 'sqrt', 'splitter': 'random'}
F1 (reported): 0.9924946382275899
Confusion Matrix: [[51 0] [1 81]]
Classification Report:

```
precision    recall  f1-score   support

          no       0.98      1.00      0.99        51
         yes       1.00      0.99      0.99        82

    accuracy                           0.99       133
   macro avg       0.99      0.99      0.99       133
weighted avg
```

Random Forest (Grid)

Best Parameters: {}:''.format(grid.best_params_),f1_macro) print("The confusion Matrix:\n",cm) print("The report:\n",clf_report) The f1_macro value for best parameter {'criterion': 'gini', 'max_features': 'log2', 'n_estimators': 50}
F1 (reported): 0.9924946382275899
Confusion Matrix: [[51 0] [1 81]]
Classification Report:

```
precision    recall  f1-score   support

          no       0.98      1.00      0.99        51
         yes       1.00      0.99      0.99        82

    accuracy                           0.99       133
   macro avg       0.99      0.99      0.99       133
weighted avg
```

SVM (Grid) (alt)

Best Parameters:
F1 (reported): None
Confusion Matrix:
Classification Report:

```
Not found.
```

3) Final model selection & justification

The final selected model: SVM (Grid).

Reasons:
- It achieved the highest reported F1 score (F1 = 0.9925) during Grid Search CV.
- When available, ROC AUC was excellent (1.00 for SVM Grid), indicating strong separability.
- The confusion matrix and classification report show very high precision and recall for both classes, indicating low false positives and false negatives on the held-out test set.

Caveats & notes:
- The dataset appears to be balanced enough in the test split (51 no / 82 yes). Still, please validate via cross-validation with more repeats or a stratified CV for production readiness.
- Multiple models (SVM, Decision Tree, Random Forest) show nearly identical test metrics — differences are minor. If interpretability is required, prefer Decision Tree / Random Forest; if raw predictive performance is priority, SVM is selected here.

Appendix: Full original Grid Search outputs (excerpt)

```
Classification Using Grid Search:

Logistic Grid Classification:

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

dataset=pd.read_csv("CKD.csv")

indep1=dataset.drop(columns=['classification'])
dep=dataset['classification']
indep = pd.get_dummies(indep1, drop_first=True)

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(indep, dep, test_size = 1/3, random_state = 0)

from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)


from sklearn.model_selection import GridSearchCV
# Define the parameter grid
param_grid = {
    'C': [0.01, 0.1, 1, 10, 100],            # Regularization strength
    'penalty': ['l1', 'l2', 'elasticnet', 'none'],  # Penalty type
    'solver': ['lbfgs', 'liblinear', 'saga'],       # Solvers compatible with penalties
}



from sklearn.linear_model import LogisticRegression


grid = GridSearchCV(LogisticRegression(), param_grid, refit = True, verbose = 3,n_jobs=-1,scoring='f1_weighted')
grid.fit(X_train, y_train)

re=grid.cv_results_
#print(re)
grid_predictions = grid.predict(X_test)


from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, grid_predictions)


from sklearn.metrics import f1_score
f1_macro=f1_score(y_test,grid_predictions,average='weighted')
print("The f1_macro value for best parameter {}:".format(grid.best_params_),f1_macro)
```

```
print("The confusion Matrix:\n",cm)


from sklearn.metrics import roc_auc_score

roc_auc_score(y_test,grid.predict_proba(X_test)[:,1])

table=pd.DataFrame.from_dict(re)


# print classification report
from sklearn.metrics import classification_report
clf_report = classification_report(y_test, grid_predictions)

print(clf_report)

The f1_macro value for best parameter {'C': 10, 'penalty': 'l2', 'solver': 'lbfgs'}: 0.9850141736106648
The confusion Matrix:
 [[51  0]
 [ 2 80]]
              precision    recall  f1-score   support

          no       0.96      1.00      0.98        51
         yes       1.00      0.98      0.99        82

    accuracy                           0.98       133
   macro avg       0.98      0.99      0.98       133
weighted avg       0.99      0.98      0.99       133

*******************************************************************************

SVM Grid:

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

dataset=pd.read_csv("CKD.csv")

indep1=dataset.drop(columns=['classification'])
dep=dataset['classification']
indep = pd.get_dummies(indep1, drop_first=True)

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(indep, dep, test_size = 1/3, random_state = 0)

from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)


from sklearn.model_selection import GridSearchCV
```

```
# Define the parameter grid
param_grid = {
    'C': [0.01, 0.1, 1, 10, 100],           # Regularization parameter
    'kernel': ['linear', 'poly', 'rbf', 'sigmoid'],  # Kernel types
    'gamma': ['scale', 'auto'],             # Kernel coefficient
    'degree': [2, 3, 4],                    # Used only for 'poly' kernel
}



from sklearn.svm import SVC

grid = GridSearchCV(
    estimator=SVC(probability=True),
    param_grid=param_grid,
    refit=True,
    verbose=3,
    n_jobs=-1,
    scoring='f1_weighted'
)
grid.fit(X_train, y_train)

re=grid.cv_results_
#print(re)
grid_predictions = grid.predict(X_test)


from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, grid_predictions)


from sklearn.metrics import f1_score
f1_macro=f1_score(y_test,grid_predictions,average='weighted')
print("The f1_macro value for best parameter {}:".format(grid.best_params_),f1_macro)

print("The confusion Matrix:\n",cm)


from sklearn.metrics import roc_auc_score

# ROC-AUC Score (for binary classification only)
try:
    roc = roc_auc_score(y_test, grid.predict_proba(X_test)[:, 1])
    print("\nROC AUC Score: {:.4f}".format(roc))
except Exception as e:
    print("\nROC AUC could not be computed (likely non-binary classification).")

table=pd.DataFrame.from_dict(re)


# print classification report
from sklearn.metrics import classification_report
clf_report = classification_report(y_test, grid_predictions)
```

```
print(clf_report)

The f1_macro value for best parameter {'C': 10, 'degree': 2, 'gamma': 'scale', 'kernel': 'sigmoid'}: 0.9924946382275899
The confusion Matrix:
 [[51  0]
 [ 1 81]]

ROC AUC Score: 1.0000
              precision    recall  f1-score   support

          no       0.98      1.00      0.99        51
         yes       1.00      0.99      0.99        82

    accuracy                           0.99       133
   macro avg       0.99      0.99      0.99       133
weighted avg       0.99      0.99      0.99       133

************************************************************
DC_Grid

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

dataset=pd.read_csv("CKD.csv")

indep1=dataset.drop(columns=['classification'])
dep=dataset['classification']
indep = pd.get_dummies(indep1, drop_first=True)

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(indep, dep, test_size = 1/3, random_state = 0)

from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

from sklearn.tree import DecisionTreeClassifier

from sklearn.model_selection import GridSearchCV

param_grid = {'criterion':['gini','entropy'],
              'max_features': ['auto','sqrt','log2'],
              'splitter':['best','random']}
■■■
■■■
grid = GridSearchCV(DecisionTreeClassifier(), param_grid, refit = True, verbose = 3,n_jobs=-1,scoring='f1_weighted')

# fitting the model for grid search
grid.fit(X_train, y_train)
re=grid.cv_results_
#print(re)
```

```
grid_predictions = grid.predict(X_test)


from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, grid_predictions)



# print classification report
from sklearn.metrics import classification_report
clf_report = classification_report(y_test, grid_predictions)

from sklearn.metrics import f1_score
f1_macro=f1_score(y_test,grid_predictions,average='weighted')
print("The f1_macro value for best parameter {}:".format(grid.best_params_),f1_macro)

print("The confusion Matrix:\n",cm)

print("The report:\n",clf_report)

The f1_macro value for best parameter {'criterion': 'entropy', 'max_features': 'sqrt', 'splitter': 'random'}: 0.992494638227589
The confusion Matrix:
 [[51  0]
 [ 1 81]]
The report:
               precision    recall  f1-score   support

          no       0.98      1.00      0.99        51
         yes       1.00      0.99      0.99        82

    accuracy                           0.99       133
   macro avg       0.99      0.99      0.99       133
weighted avg       0.99      0.99      0.99       133

**********************************************************************
4.RF Grid:

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

dataset=pd.read_csv("CKD.csv")

indep1=dataset.drop(columns=['classification'])
dep=dataset['classification']
indep = pd.get_dummies(indep1, drop_first=True)

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(indep, dep, test_size = 1/3, random_state = 0)

from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
```

```
X_test = sc.transform(X_test)

from sklearn.ensemble import RandomForestClassifier

from sklearn.model_selection import GridSearchCV

param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_features': ['sqrt', 'log2'],  # 'auto' deprecated
    'n_estimators': [10, 50, 100],
}
■■■
■■■
grid = GridSearchCV(RandomForestClassifier(), param_grid, refit = True, verbose = 3,n_jobs=-1,scoring='f1_macro')

# fitting the model for grid search
grid.fit(X_train, y_train)


re=grid.cv_results_
#print(re)
grid_predictions = grid.predict(X_test)


from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, grid_predictions)



# print classification report
from sklearn.metrics import classification_report
clf_report = classification_report(y_test, grid_predictions)

from sklearn.metrics import f1_score
f1_macro=f1_score(y_test,grid_predictions,average='weighted')
print("The f1_macro value for best parameter {}:".format(grid.best_params_),f1_macro)

print("The confusion Matrix:\n",cm)

print("The report:\n",clf_report)

The f1_macro value for best parameter {'criterion': 'gini', 'max_features': 'log2', 'n_estimators': 50}: 0.9924946382275899
The confusion Matrix:
 [[51  0]
 [ 1 81]]
The report:
               precision    recall  f1-score   support

          no       0.98      1.00      0.99        51
         yes       1.00      0.99      0.99        82

    accuracy                           0.99       133
   macro avg       0.99      0.99      0.99       133
```

```
weighted avg       0.99      0.99      0.99       133

**********************************************************************

5.SVM Grid


import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

dataset=pd.read_csv("CKD.csv")

indep1=dataset.drop(columns=['classification'])
dep=dataset['classification']
indep = pd.get_dummies(indep1, drop_first=True)

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(indep, dep, test_size = 1/3, random_state = 0)

from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

from sklearn.svm import SVC

from sklearn.model_selection import GridSearchCV

param_grid = {'kernel':['linear','rbf','poly','sigmoid'],
              'gamma':['auto','scale'],
              'C':[10,100,1000,2000,3000]}



grid = GridSearchCV(SVC(probability=True), param_grid, refit = True, verbose = 3,cv=5,n_jobs=-1,scoring='f1_weighted')

# fitting the model for grid search
grid.fit(X_train, y_train)

re=grid.cv_results_
#print(re)
grid_predictions = grid.predict(X_test)


from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, grid_predictions)



# print classification report
from sklearn.metrics import classification_report
clf_report = classification_report(y_test, grid_predictions)
```

```
from sklearn.metrics import f1_score
f1_macro=f1_score(y_test,grid_predictions,average='weighted')
print("The f1_macro value for best parameter {}:".format(grid.best_params_),f1_macro)

print("The confusion Matrix:\n",cm)

print("The report:\n",clf_report)
```

```
The f1_macro value for best parameter {'C': 10, 'gamma': 'auto', 'kernel': 'sigmoid'}: 0.9924946382275899
The confusion Matrix:
 [[51  0]
 [ 1 81]]
The report:
               precision    recall  f1-score   support

           no       0.98      1.00      0.99        51
          yes       1.00      0.99      0.99        82

    accuracy                           0.99       133
   macro avg       0.99      0.99      0.99       133
weighted avg       0.99      0.99      0.99       133
```